Structured Program Development
(C Programming Language)

Dr. Thien Huynh-The

Dept. Comp. Commun. Eng.

HCMC Technology and Education

Content

- Introduction to algorithms and pseudocode
- Control structure
- The if selection statement
- The if...else selection statement
- The while repetition statement
- Formulating Algorithms: Case Study 1 (Counter-Controlled Repetition)
- Formulating Algorithms with Top-Down, Stepwise Refinement: Case Study 2 (Sentinel-Controlled Repetition)
- Formulating Algorithms with Top-Down, Stepwise Refinement: Case Study 3 (Nested Control Structures)
- Operators: assignment, increment, and decrement

Introduction

- Before writing a program
  - Have a through understanding of the problem
  - Carefully plan an approach/method/algorithm to solve it
- While writing a program
  - Know what “building blocks” are available
  - Use good programming principles

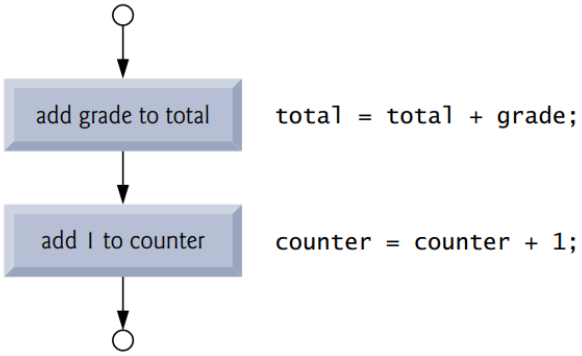
Algorithms

- Computing problems
  - All can be solved by executing a series of actions in a specific order
- Algorithm: procedure in terms of
  - Actions to be executed
  - The order in which these actions are to be executed
- Program control
  - Specify order in which statements are to be executed

Pseudo Code

- Pseudocode
  - Artificial, informal language that helps us develop algorithms
  - Similar to everyday English
  - Not actually executed on computers
  - Helps us “think out” a program before writing it
    - Easy to convert into a corresponding C++ program
    - Consists only of executable statements

Control Structures



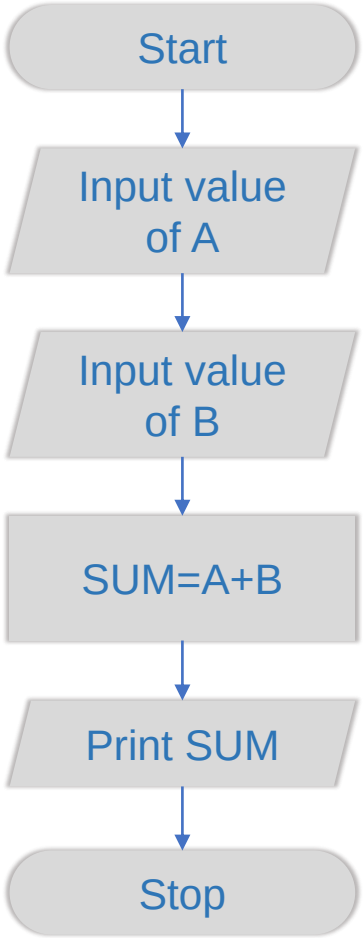
- Sequential execution
  - Statements executed one after the other in the order written
- Transfer of control
  - When the next statement executed is not the next one in sequence
  - Overuse of goto statements led to many problems
- Bohm and Jacopini
  - All programs written in terms of 3 control structures
    - Sequence structures: Built into C. Programs executed sequentially by default
    - Selection structures: C has three types: if, if…else, and switch
    - Repetition structures: C has three types: while, do…while and for

Control Structures

- Flowchart
  - Graphical representation of an algorithm
  - Drawn using certain special-purpose symbols connected by arrows called flowlines
  - Rectangle symbol (action symbol):
    - Indicates any type of action
  - Oval symbol:
    - Indicates the beginning or end of a program or a section of code
  - Diamonds: Decision or switching
  - Small circles: Connector
- Single-entry/single-exit control structures
  - Connect exit point of one control structure to entry point of the next (control-structure stacking)
  - Makes programs easy to build

Algorithm – Pseudo code - Flowchart

Algorithm	Pseudo Code
Step 1: Start	Begin
Step 2: Input the first number A	Input A
Step 3: Input the second number B	Input B
Step 4: Calculate SUM=A+B	Compute SUM=A+B
Step 5: Display SUM	Print SUM
Step 6: Stop	End



If Selection Statement

- Selection structure:
  - Used to choose among alternative courses of action
  - Pseudocode:

```
If student's grade is greater than or equal to 60
   Print "Passed"
```

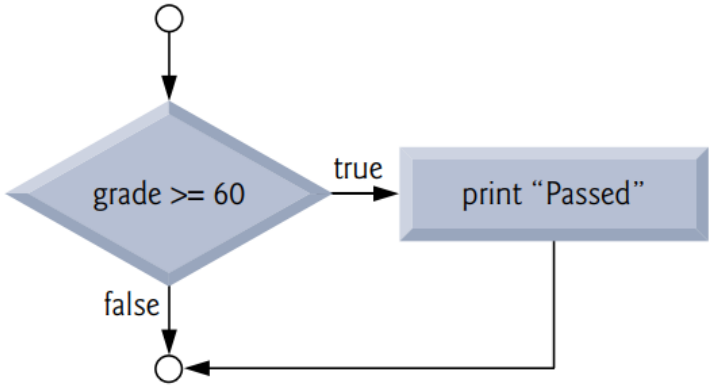
- If condition true
  - Print statement executed and program goes on to next statement
  - If false, print statement is ignored and the program goes onto the next statement
  - Indenting makes programs easier to read
    - C ignores whitespace characters

If Selection Statement

- Pseudocode statement in C:

```
if ( grade >= 60 )
   printf( "Passed\n" );
```

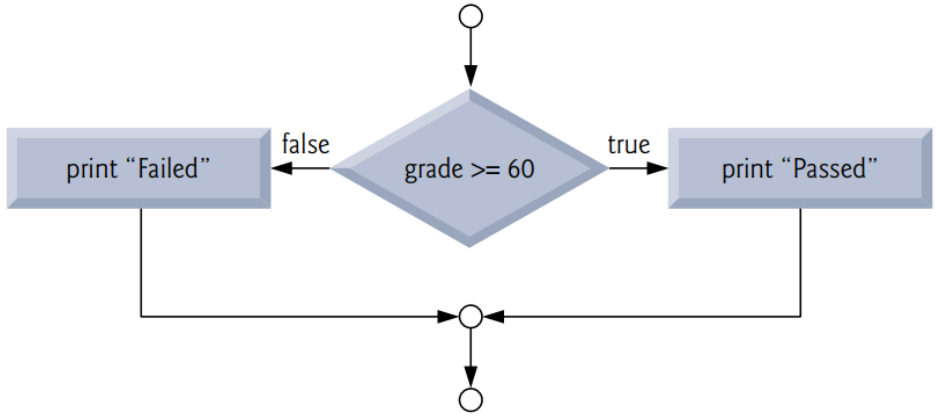
  - C code corresponds closely to the pseudocode
- Diamond symbol (decision symbol)
  - Indicates decision is to be made
  - Contains an expression that can be true or false
  - Test the condition, follow appropriate path



If Selection Statement

- `if`
  - Only performs an action if the condition is true
- `if…else`
  - Specifies an action to be performed both when the condition is true and when it is false
- Psuedocode:

```
If student's grade is greater than or equal to 60
   Print "Passed"
else
   Print "Failed"
```



If...else Selection Statement

- Tips:
  - Indent both body statements of an `if...else` statement.
  - If there are several levels of indentation, each level should be indented the same additional amount of space.
- The preceding pseudocode `if...else` statement may be written in C

```
if ( grade >= 60 ) {
   printf( "Passed\n" );
} //end if
else {
   printf( "Failed\n" );
} //end else
```

If...else Selection Statement

- Ternary conditional operator (?:)
  - Takes three arguments (condition, value if true, value if false)
  - Our pseudocode could be written:

```
printf( "%s\n", grade >= 60 ? "Passed" : "Failed" );
```

  - Or it could have been written:

```
grade >= 60 ? printf( "Passed\n" ) : printf( "Failed\n" );
```

- Nested `if…else` statements
  - Test for multiple cases by placing `if…else` selection statements inside `if…else` selection statement
  - Once condition is met, rest of statements skipped
  - Deep indentation usually not used in practice

Tip: use expressions of the same type for the second and third operands of the conditional operator (?:) to avoid subtle errors.

If...else Selection Statement

```
If student’s grade is greater than or equal to 90
   Print “A”
else
   If student’s grade is greater than or equal to 80
      Print “B”
   else
      If student’s grade is greater than or equal to 70
         Print “C”
      else
         If student’s grade is greater than or equal to 60
            Print “D”
         else
            Print “F”
```

```
If ( grade >=90 ){
    printf( "A" );
} // end if
else {
    if (grade >= 80){
        printf( "B" );
    } // end if
    else {
        if (grade >= 70){
            printf( "C" );
        } // end if
        else {
            if (grade >= 60){
                printf( "D" );
            } // end if
             else {
                   printf( "F"
);
             } // end else
         } // end else
    } // end else
} // end else
```

If...else Selection Statement

- Compound statement:
  - Set of statements within a pair of braces
  - Example:

```
if ( grade >= 60 )
   printf( "Passed.\n" );
else {
   printf( "Failed.\n" );
   printf( "You must take this course
       again.\n" );
   }
```

  - Without the braces, the statement `printf( "You must take this course  again.\n" );` would be executed automatically

If...else Selection Statement

- Notes:
  - A compound statement can be placed anywhere in a program that a single statement can be placed.
  - Forgetting one or both of the braces that delimit a compound statement.
- Block:
  - Compound statements with declarations
- Syntax errors
  - Caught by compiler
- Logic errors:
  - Have their effect at execution time
  - Non-fatal: program runs, but has incorrect output
  - Fatal: program exits prematurely

Assignment 1 (2-1)

- Write a program to print the tax order and tax percentage with income entered by user using `if…else` statement.

Order	Income per year (M)	Tax (%)
1	Up to 60	5
2	From 60 up to 120	10
3	From 120 up to 216	15
4	From 216 up to 384	20
5	From 384 up to 624	25
6	From 624 up to 960	30
7	Over 960	35

While Repetition Statement

- Repetition structure
  - Programmer specifies an action to be repeated while some condition remains true
  - Pseudocode:

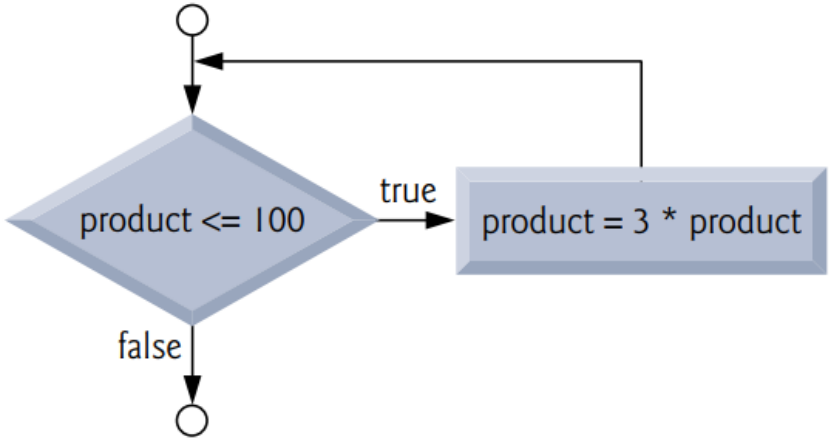
```
While there are more items on my shopping list
   Purchase next item and cross it off my list
```

  - While loop repeated until condition becomes false
- Example:

```
int product = 2;
while ( product <= 1000 )
    product = 2 * product;
```

While Repetition Statement

- Errors:
  - Not providing the body of a while statement with an action that eventually causes the condition in the while to become false. Normally, such a repetition structure will never terminate—an error called an “infinite loop”.
  - Spelling the keyword `while` with an uppercase W as in `While`



Counter-Controlled Repetition

- Formulating Algorithms Case Study 1: Counter-controlled repetition
  - Loop repeated until counter reaches a certain value
  - Definite repetition: number of repetitions is known
  - Example: A class of ten students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Determine the class average on the quiz.
- Pseudocode

```
Set total to zero
Set grade counter to one
While grade counter is less than or equal to ten
    Input the next grade
    Add the grade into the total
    Add one to the grade counter
Set the class average to the total divided by ten
Print the class average
```

Counter-Controlled Repetition

```c
// Fig. 3.6: fig03_06.c
// Class average program with counter-controlled iteration.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   unsigned int counter; // number of grade to be entered next
   int grade; // grade value
   int total; // sum of grades entered by user
   int average; // average of grades

   // initialization phase
   total = 0; // initialize total
   counter = 1; // initialize loop counter

   // processing phase
   while ( counter <= 10 ) { // loop 10 times
      printf( "%s", "Enter grade: " ); // prompt for input
      scanf( "%d", &grade ); // read grade from user
      total = total + grade; // add grade to total
      counter = counter + 1; // increment counter
   } // end while

   // termination phase
   average = total / 10; // integer division

   printf( "Class average is %d\n", average ); // display result
} // end function main
```

Counter to control while loop

Initialize counter to 1

While loop iterates as long as counter <=10

Calculate the total grade

Increase the counter

Calculate the average

- If a counter or total is not initialized, the results will probably be incorrect (a logic error).
- Initialize all counters and totals.

Exercise

- Ex 2-2: Viết chương trình C nhập điểm của N sinh viên (với N được nhập từ bàn phím và là số nguyên dương). Tính và in ra điểm trung bình của N sinh viên đó.
  - Yêu cầu nhập N. Kiểm tra tính hợp lệ của N
  - Bộ đếm biết trước
  - Biến tổng điểm bằng 0 lúc khởi tạo
- Ex 2-3: Viết chtrình C in ra bảng cửu chương

Exercise

- Ex 2-4: Viết chtrình C theo yêu cầu sau
  - Nhập vào số tiền cho trước lớn hơn 100 usd. Yêu cầu nhập lại nếu không thỏa
  - Thực hiện mua 10 món hàng với giá trị mỗi món nhỏ hơn hoặc bằng 10 usd. Nếu nhập giá trị món hang lớn hơn 10 thì yêu cầu nhập lại
  - Tính và in ra tiền thừa
  - Tính và in ra giá trị trung bình của 10 món hàng
- Ex 2-5: Viết chtrình C theo yêu cầu sau
  - Nhập điểm cho sinh viên có trong danh sách (không biết trước có bao nhiêu sv). Dừng việc nhập điểm bằng cách nhậo một số đặc biệt nào đó.
  - Tính điểm trung bình của các sv có điểm lớn hơn hoặc bằng 5.

Assignment 2

3.23 *(Find the Largest Number)* The process of finding the largest number (i.e., the maximum of a group of numbers) is used frequently in computer applications. For example, a program that determines the winner of a sales contest would input the number of units sold by each salesperson. The salesperson who sells the most units wins the contest. Write a pseudocode program and then a program that inputs a series of 10 non-negative numbers and determines and prints the largest of the numbers. [*Hint:* Your program should use three variables as shown below.]

`counter:`	A counter to count to 10 (i.e., to keep track of how many numbers have been input and to determine when all 10 numbers have been processed)
`number:`	The current number input to the program
`largest:`	The largest number found so far

3.25 *(Tabular Output)* Write a program that utilizes looping to produce the following table of values:

```
A       A+3     A+6     A+9

7       10      13      63
14      17      20      126
21      24      27      189
28      31      34      252
35      38      41      315
```

Quiz

3.28 What does the following program print?

```
#include <stdio.h>
int main( void )

{
   int x = 1;
   int y = 0;
   while ( x <= 5 ) { // loop 5 times
      total+= x * x; // perform calculation
      printf( "%d\n", x*x);
      ++x; // increment counter x
   } // end while
   printf( "Total is %d\n", total); // display result
} // end function main
```

Quiz

3.29 What does the following program print?

```
#include <stdio.h>

int main( void )
{
   int outer_count = 1; // initialize count
   while ( outer_count <= 10 ) { // loop 10 times
      int inner_count = 1;
      while ( inner_count <= outer_count ) {
         printf( "*" );
         inner_count++;
      } // end inner while
      printf( "\n" );
      outer_count++;
   } // end outer while
} // end main
```

Sentinel-Controlled Iteration

- Formulating algorithms with top-down, stepwise refinement case study with sentinel-controlled iteration
- Problem becomes:

  Develop a class-averaging program that will process an arbitrary number of grades each time the program is run.

  - Unknown number of students
  - How will the program know to end?
- Use sentinel value
  - Also called signal value, dummy value, or flag value
  - Indicates “end of data entry.”
  - Loop ends when user inputs the sentinel value
  - Sentinel value chosen so it cannot be confused with a regular input (such as -1 in this case)

Sentinel-Controlled Iteration

- Top-down, stepwise refinement
  - Begin with a pseudocode representation of the top:

```
Determine the class average for the quiz
```

  - Divide top into smaller tasks and list them in order:

```
Initialize variables
Input, sum and count the quiz grades
Calculate and print the class average
```

- Many programs have three phases:
  - Initialization: initializes the program variables
  - Processing: inputs data values and adjusts program variables accordingly
  - Termination: calculates and prints the final results

- Errors: Choosing a sentinel value that is also a legitimate data value.
- Notes: Each refinement, as well as the top itself, is a complete specification of the algorithm; only the level of detail varies.

Sentinel-Controlled Iteration

- Refine the initialization phase from Initialize variables to:

```
Initialize total to zero
Initialize counter to zero
```

- Refine Input, sum and count the quiz grades to

```
Input the first grade (possibly the sentinel)
While the user has not as yet entered the sentinel
    Add this grade into the running total
    Add one to the grade counter
    Input the next grade (possibly the sentinel)
```

- Refine Calculate and print the class average to

```
If the counter is not equal to zero
    Set the average to the total divided by the counter
    Print the average
else
    Print “No grades were entered”
```

- Note: An attempt to divide by zero causes a fatal error.

Sentinel-Controlled Iteration

```c
// Fig. 3.8: fig03_08.c
// Class-average program with sentinel-controlled iteration.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   unsigned int counter; // number of grades entered
   int grade; // grade value
   int total; // sum of grades

   float average; // number with decimal point for average

   // initialization phase
   total = 0; // initialize total
   counter = 0; // initialize loop counter

   // processing phase
   // get first grade from user
   printf( "%s", "Enter grade, -1 to end: " ); // prompt for input
   scanf( "%d", &grade ); // read grade from user

   // loop while sentinel value not yet read from user
   while ( grade != -1 ) {
      total = total + grade; // add grade to total
      counter = counter + 1; // increment counter

      // get next grade from user
      printf( "%s", "Enter grade, -1 to end: " ); // prompt for input
      scanf("%d", &grade); // read next grade
   } // end while
```

Float type indicates variable can be a non-integer

while loop will repeat following statements until user enters a value −1.

Sentinel-Controlled Iteration

```
33      // termination phase
34      // if user entered at least one grade
35      if ( counter != 0 ) {
36
37         // calculate average of all grades entered
38         average = ( float ) total / counter; // avoid truncation
39
40         // display average with two digits of precision
41         printf( "Class average is %.2f\n", average );
42      } // end if
43      else { // if no grades were entered, output message
44         puts( "No grades were entered" );
45      } // end else
46   } // end function main
```

Ensure user enter at least one grade (avoid devision-by-zero error)

Convert total from int to float type

Print result with 2 digits after decimal point

Assignment 2 (2-6)

3.16 *(Sales Tax)* Sales tax is collected from buyers and remitted to the government. A retailer has to file a monthly sales tax report which lists the sales for the month and the amount of sales tax collected, at both the county and state levels. Develop a program that will input the total collections for a month, calculate the sales tax on the collections, and display the county and state taxes. Assume that states have a 4% sales tax and counties have a 5% sales tax. Here is a sample input/output dialog.

```
Enter total amount collected (-1 to quit): 45678
Enter name of month: January
Total Collections: $ 45678.00
Sales: $ 41906.42
County Sales Tax: $ 2095.32
State Sales Tax: $ 1676.26
Total Sales Tax Collected: $ 3771.58

Enter total amount collected (-1 to quit): 98000
Enter name of month: February
Total Collection: $ 98000
Sales: $ 89908.26
County Sales Tax: $ 4495.41
State Sales Tax: $ 3596.33
Total Sales Tax Collected: $ 8091.74
Enter total amount collected (-1 to quit): -1
```

Exercise (2-7)

- Viết chtrình C:
  - Yêu cầu nhập nào N sinh viên
  - Với từng sinh viên, yêu cầu nhập vào kết quả: nhập số 1 nếu đậu và số 2 nếu rớt
  - Sau khi nhập hết N sinh viên thì tổng kết
    - In ra bao nhiêu sinh viên đậu
    - In ra bao nhiêu sinh viên rớt
    - Nếu có từ 80% sv đậu thì sẽ in ra “Khen thuong!”

Nested Control Structures

- Formulating algorithms with top-down, stepwise refinement case study with nested control structures
- Problem
  - A college has a list of test results (1 = pass, 2 = fail) for 10 students
  - Write a program that analyzes the results
    - If more than 8 students pass, print "Raise Tuition"
- Notice that
  - The program must process 10 test results
    - Counter-controlled loop will be used
  - Two counters can be used
    - One for number of passes, one for number of fails
  - Each test result is a number—either a 1 or a 2
    - If the number is not a 1, we assume that it is a 2

Nested Control Structures

- Top level outline
  - Analyze exam results and decide if tuition should be raised
- First Refinement
  - Initialize variables
  - Input the ten quiz grades and count passes and failures
  - Print a summary of the exam results and decide if tuition should be raised
- Refine Initialize variables to
  - Initialize passes to zero
  - Initialize failures to zero
  - Initialize student counter to one

Nested Control Structures

- Refine Input the ten quiz grades and count passes and failures to

```
While student counter is less than or equal to ten
    Input the next exam result
    If the student passed
            Add one to passes
    else
            Add one to failures

    Add one to student counter
```

- Refine Print a summary of the exam results and decide if tuition should be raised to

```
Print the number of passes
Print the number of failures
If more than eight students passed
    Print “Raise tuition”
```

Nested Control Structures

Pseudocode

```
Initialize passes to zero
Initialize failures to zero
Initialize student to one
While student counter is less than or equal to ten
    Input the next exam result
    If the student passes
        Add one to passes
    Else
        Addone to failure
    Add one to student counter
Print the number of passes
Print the number of failures
If more than eight students passes
    Print “Raise tuition”
```

Nested Control Structures

```c
// Fig. 3.10: fig03_10.c
// Analysis of examination results.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   // initialize variables in definitions
   unsigned int passes = 0; // number of passes
   unsigned int failures = 0; // number of failures
   unsigned int student = 1; // student counter
   int result; // one exam result

   // process 10 students using counter-controlled loop
   while ( student <= 10 ) {

      // prompt user for input and obtain value from user
      printf( "%s", "Enter result ( 1=pass,2=fail ): " );
      scanf( "%d", &result );

      // if result 1, increment passes
      if ( result == 1 ) {
         passes = passes + 1;
      } // end if
      else { // otherwise, increment failures
         failures = failures + 1;
      } // end else

      student = student + 1; // increment student counter
   } // end while
```

while loop continues until 10 students have been processed

if and else statements are nested inside while loop

```c

    // termination phase; display number of passes and failures
    printf( "Passed %u\n", passes );
    printf( "Failed %u\n", failures );

    // if more than eight students passed, print "Bonus to instructor!"
    if ( passes > 8 ) {
       puts( "Bonus to instructor!" );
    } // end if
 } // end function main
```

```
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Passed 6
Failed 4
```

```
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Passed 9
Failed 1
Bonus to instructor!
```

Assignment Operators

- Assignment operators abbreviate assignment expressions

  `c = c + 3;`

  can be abbreviated as `c += 3`; using the addition assignment operator

- Statements of the form

  `variable = variable operator expression;`

  can be rewritten as

  `variable operator= expression;`

- Examples of other assignment operators:

```
d -= 4      (d = d - 4)
e *= 5      (e = e * 5)
f /= 3      (f = f / 3)
g %= 9      (g = g % 9)
```

Assignment Operators

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Fig. 3.11 | Arithmetic assignment operators.

Increment and Decrement Operators

- Increment operator (++)
  - Can be used instead of `c+=1`
- Decrement operator (--)
  - Can be used instead of `c-=1`
- Pre-increment
  - Operator is used before the variable (`++c` or `--c`)
  - Variable is changed before the expression it is in is evaluated
- Post-increment
  - Operator is used after the variable (`c++` or `c--`)
  - Expression executes before the variable is changed

- If `c` equals 5, then

```
printf( "%d", ++c );
```

  - Prints 6

```
printf( "%d", c++ );
```

  - Prints 5
  - In either case, `c` now has the value of 6
- When variable not in an expression
  - Preincrementing and postincrementing have the same effect

```
++c;
printf( “%d”, c );
```

  - Has the same effect as

```
c++;
printf( “%d”, c );
```

Increment and Decrement Operators

Operator	Sample expression	Explanation
`++`	`++a`	Increment `a` by 1, then use the new value of `a` in the expression in which `a` resides.
`++`	`a++`	Use the current value of `a` in the expression in which `a` resides, then increment `a` by 1.
`--`	`--b`	Decrement `b` by 1, then use the new value of `b` in the expression in which `b` resides.
`--`	`b--`	Use the current value of `b` in the expression in which `b` resides, then decrement `b` by 1.

Fig. 3.12 | Increment and decrement operators

Increment and Decrement Operators

- Avoid using statements with increment or decrement operators
- Attempting to use the increment or decrement operator on an expression other than a simple variable name is a syntax error, e.g., writing `++(x + 1)`.

```c
// Fig. 3.13: fig03_13.c
// Preincrementing and postincrementing.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   int c; // define variable

   // demonstrate postincrement
   c = 5; // assign 5 to c
   printf( "%d\n", c ); // print 5
   printf( "%d\n", c++ ); // print 5 then postincrement
   printf( "%d\n\n", c ); // print 6

   // demonstrate preincrement
   c = 5; // assign 5 to c
   printf( "%d\n", c ); // print 5
   printf( "%d\n", ++c ); // preincrement then print 6
   printf( "%d\n", c ); // print 6
} // end function main
```

c is printed, then increasing

c is increased, then printing

Increment and Decrement Operators

Operators	Associativity	Type
++ *(postfix)* -- *(postfix)*	right to left	postfix
+ - (*type*) ++ *(prefix)* -- *(prefix)*	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

Fig. 3.14 | Precedence and associativity of the operators encountered so far in the text.

Assignment 3

- Viết chtrình tính bthức sau với số lượng thành phần n là số nguyên dương nhập từ bàn phím

$$\frac{1}{1}+\frac{1}{3}+\frac{1}{5}+\frac{1}{7}+\ldots$$

- Viết chtrình tính bthức sau với số lượg thành phầnn là số nguyên dương lớn hơn 2 được nhập từ bàn phím

$$1!+2!+3!+4!+\cdots$$

Assignment 4

3.45 *(Factorial)* The factorial of a nonnegative integer n is written $n!$ (pronounced "n factorial") and is defined as follows:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \ldots \cdot 1 \quad \text{(for values of } n \text{ greater than or equal to 1)}$$

and

$$n! = 1 \quad \text{(for } n = 0\text{)}.$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is 120.

a) Write a program that reads a nonnegative integer and computes and prints its factorial.

b) Write a program that estimates the value of the mathematical constant e by using the formula:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \ldots$$

c) Write a program that computes the value of e^x by using the formula

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots$$